CHEMIN

Social Network for the Portuguese Way of St. James

Technical Documentation

Damián López Piñeiro

2º Multiplatform Application Development

Intermodular project 2025/2026

Contenido

1. System architecture 4
   1.1 General scheme 4
   1.2 Main technologies 5
2. API 5
   2.1 List of endpoints 6
   2.2 HTTP Methods Used 7
   2.3 Authentication 7
   2.4 Examples of requests and responses 7
      Login 7
      Upload image 8
      Register User 8
      Get Filtered Publications 8
3. Database 9
   3.1 Entity-Relationship Diagram 9
   3.2 Description of Tables and Fields 9
   3.3 Important rules 11
   3.4 Migration Management 11
4. Web application 11
   4.1 Technology used 11
   4.2 Project structure 12
   4.3 API Connection 13
5. Mobile application 14
   5.1 Technology used 14
   5.2 Project structure 14
      5.3 API Connection 16
6. Desktop application 17
   6.1 Technology used 17
   6.2 Project structure 17
   6.3 API Connection 17
7. Security 18
   7.1 Security mechanisms 18
   7.2 User and permissions management 19
   7.3 Good practices 19
8. Deployment 19
   8.1 API Deployment 19

Prerequisites 20
Steps 20
8.2 Web Application Deployment 20
Prerequisites 20
Steps 20
8.3 Mobile Application Deployment 20
Prerequisites 20
Steps 21
8.4 Desktop Application Deployment 21
Prerequisites 21
Steps 21
8.2 Execution Environments 21
9. Usage and Maintenance Guide 22
9.1 Running Locally 22
9.2 Adding Functionality 22
9.3 Common Issues 22
10. Future Improvements 23
10.1 Planned Improvements 23
10.2 New Functionalities 24
10.3 Technical Optimizations 24

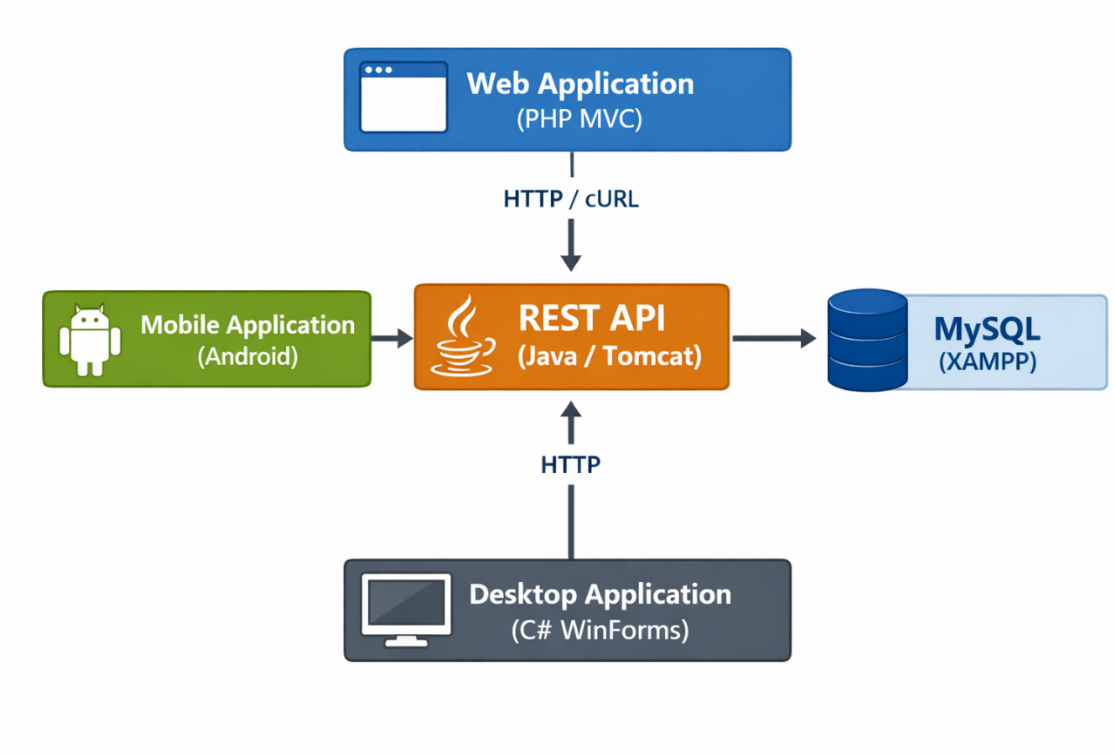
1. System architecture

Chemin is a multiplatform social network designed specifically for pilgrims on the Portuguese Way of St. James. The system follows a client-server architecture in which three client applications consume a centralized REST API that manages the system's database access.

The core idea behind Chemin is to connect pilgrims based on their geographic location along the route. Publications are automatically tagged with pilgrim's current stage of the Camino, and the feed filters content to show only posts from nearby stages. This geolocation-based filtering is what differentiates Chemin from generic social networks.

1.1 General scheme

All three client applications communicate with the REST API via HTTP requests. The API acts as a single point, ensuring data consistency across platforms. No client communicates directly with the database.



The diagram above illustrates how the three clients connect to the API. Each client uses a different mechanism to make HTTP requests:

- The web application uses PHP with cURL. All JavaScript fetch() calls go to local PHP scripts, which in turn call the API using cURL. This ensures that all requests pass through the server side.

- The Android mobile app uses Volley for most requests and HttpURLConnection for operations requiring more control, such as uploading images in Base64 format.
- The Desktop application uses C# HttpClient to make direct HTTP calls to the API, with Newtonsoft.Json handling JSON serialization and deserialization.

The geolocation feature works as follows: when a user opens the mobile app, it requests GPS permission and sends the current coordinates to the API via the /usuario/ubicacion endpoint. The API stores these coordinates in the database. When a user creates a publication (image or text), the backend looks up the user's stored coordinates and uses the Haversine formula to calculate the nearest stage of the Camino, which is then stored with the publication. When loading the feed, the API retrieves the user's current stage and returns only publications from that stage and the next one.

1.2 Main technologies

Component	Technologies
API	Java, Tomcat
Database	MySQL(XAMPP in local environment)
App Web	PHP8 + MVC
Mobile App	Android Studio(Java)
Desktop App	C#, .NET Windows Forms

2. API

The REST API is the center of the Chemin system. It centralizes all logic and serves as the only point of access to the database. It has been developed in Java using the JAX-RS specification and deployed on an Apache Tomcat 10.0.27 server. All client applications communicate with it by sending HTTP requests and receiving JSON-formatted responses.

The API also implements a CORS filter (CORSFilter.java) that allows requests from any origin, which is necessary for the web client to communicate with the backend from the browser.

List of endpoints

The API is organized into two main resource groups: users and publications. Each resource is handled by a dedicated Java class containing the corresponding endpoints.

Users

Endpoint	Method	Description
/usuario/registro	POST	Register a new user. Hashes the password with BCrypt before storing.
/usuario/login	POST	Authenticate user. Validates credentials using BCrypt.checkpw().
/usuario/actualizar	POST	Update user profile data (name, email, password, gender).
/usuario/eliminar/{id}	DELETE	Delete user and all their publications via CASCADE.
/usuario/ubicacion	POST	Update user's GPS coordinates (latitude and longitude).

Publications

Endpoint	Method	Description
/publicacion/imagen	POST	Upload an image. Calculates the pilgrim's Camino stage from GPS coordinates and stores it with the publication.
/publicacion/texto	POST	Publish a text post. Also calculates the Camino stage from GPS coordinates.
/publicacion/lista	GET	Get all publications without stage filter. Used by the Desktop app.
/publicacion/lista/{id}	GET	Get publications filtered by the user's current Camino stage and the next one.

		Used by Web and Mobile.
/publicacion/eliminar/{id}	DELETE	Delete a publication by its ID.

2.1 HTTP Methods Used

The API uses the following standard HTTP methods through its endpoints:

Method	Usage in Chemin
GET	Retrieve publications(feed, filtered feed)
POST	Create resources: register user, login, upload image, publish text, update user, update location.
DELETE	Remove resources: delete publication, delete user account.

2.3 Authentication

The API does not implement token-based authentication such as JWT. Authentication is handled via the /usuario/login endpoint, which validates the submitted credentials against the database using BCrypt to compare the submitted password with the stored hash.

Once authenticated, session management is delegated to each client application:

- Web: PHP sessions store the username and userID server-side.
- Mobile: SharedPreferences store the user ID, username, name, email and gender locally on the device.
- Desktop: A static Sesion class holds the user ID, username and name in memory during the application's runtime.

2.4 Examples of requests and responses

Login

Request POST /usuario/login:

{ "username": "damian", "contrasenha": "mi_contrasena" }

Successful response (200):

{ "id": 1, "nombreCompleto": "Damian", "username": "damian", "email": "d@d.com", "genero": "hombre" }

Error responses:

Code	Description
401	Incorrect credentials – username not found or wrong password
500	Internal server error.

Upload image

Request POST /publicacion/imagen:

{ "idUsuario": 1, "nombre": "foto.jpg", "imagen": "<base64>" }

Successful response (200):

"Imagen subida"

Error responses:

Code	Description
400	Incomplete image data – missing name or image field.
500	Error saving image to database.

Register User

Request - POST /usuario/registro:

{ "nombreCompleto": "Damian", "username": "damian", "email": "d@d.com", "contrasenha": "pass", "genero": "hombre" }

Successful response (201):

{ "status": "ok", "message": "Usuario registrado" }

Get Filtered Publications

Request – GET /publicacion/lista/1 (where 1 is the user ID):

Successful response (200) – returns an array of the publications from the user's current Camino stage and the next:

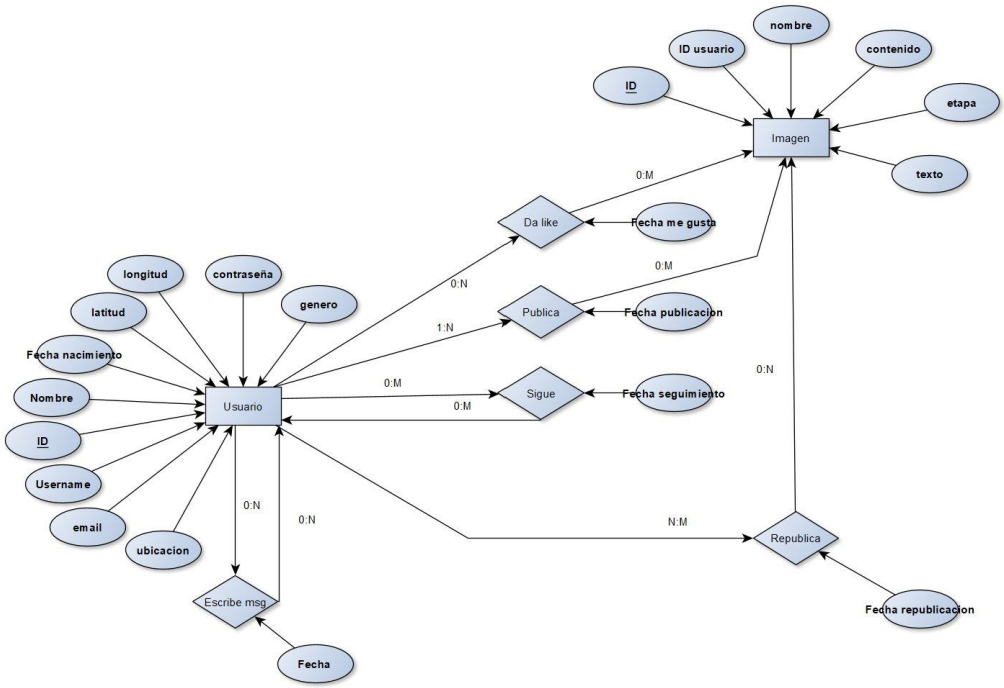
[{ "id": 5, "idUsuario": 1, "username": "damian", "texto": "Buenas vistas", "imagen": "<base64>", "etapa": "Caldas de Reis" }]

3. Database

Chemin uses a MySQL relational database managed through XAMPP. Database access is performed from the Java API using JDBC with PreparedStatements, which prevents SQL injection attacks by separating query structure from user-supplied data.

The database schema is structured around two active user tables (usuario and imagen) and six additional tables that are already created to support future features such as messaging, likes, reposts and followers. All tables use InnoDB engine with utf8mb4 character set for full Unicode support.

3.1 Entity-Relationship Diagram



The diagram shows the complete data model including both implemented relationships (Usuario - Imagen) and planned relationships (likes, followers, messages, reposts). The imagen entity acts as the central publication table, storing both image and text posts along with the Camino stage where they were published.

3.2 Description of Tables and Fields

Table: usuario

Stores all registered user data. The latitude and longitude fields are updated from the mobile app each time the user opens the application, enabling the stage-based publication filtering.

Field	Type	Restrictions	Description
Id_usuario	INT	PK, AUTO_INCREMENT	Unique Identifier
nombre	VARCHAR(100)	NOT NULL	Full name
username	VARCHAR(50)	UNIQUE, NOT NULL	Full name
email	VARCHAR(100)	UNIQUE, NOT NULL	email
contrasenha	VARCHAR(255)	NOT NULL	password
genero	VARCHAR(20)	NULL	User gender
latitude	DOUBLE	NULL	Mobile GPS latitude
longitude	DOUBLE	NULL	Mobile GPS length
location	VARCHAR(100)	NULL	Not currently used

Table: imagen

The central publication table. Despite its name, it stores both image publications and text-only posts. The etapa field is automatically calculated by the backend based on the user's GPS coordinates at the time of posting, using the Haversine algorithm to find the nearest stage of the Portuguese Camino.

Field	Type	Restrictions	Description
Id	INT	PK, AUTO_INCREMENT	Unique Identifier
Id_usuario	INT	FK usuario CASCADE	User who posted
nombre	VARCHAR(255)	NOT NULL	File name
contenido	LONGBLOB	NULL	Image bytes
texto	TEXT	NULL	Text content
etapa	VARCHAR(100)	NULL	Stage of the Camino

Table for future functionalities

Table	Purpose
publicacion	Dated posts
texto	Text from publications (separate from the image table)
mensaje	Private messages between users
like_publicacion	Like system
republica	Republications
seguidor	Follower system

3.3 Important rules

- Passwords are always stored hashed using BCrypt, never in plain text.
- The image table has a foreign key (FK) for users with DELETE CASCADE, deleting posts when a user is deleted.
- The stage field is automatically calculated in the backend using the Haversine on the user's GPS coordinates
- The latitude and longitude fields are updated from the mobile app upon login.
- The username and the email fields have a UNIQUE restriction to prevent duplicates.

3.4 Migration Management

No automated migration system is used. Database schema changes are applied manually using SQL statements in XAMPP'S phpMyAdmin. This approach was chosen for simplicity.

During development, the schema evolved using ALTER TABLE statements as new features were added:

- ALTER TABLE imagen ADD COLUMN texto TEXT NULL
  It is used to support text-only posts.
- ALTER TABLE imagen ADD COLUMN etapa VARCHAR(100) NULL
  Added for Camino stage filtering.
- ALTER TABLE usuario ADD COLUMN latitud DOUBLE NULL
  Added for GPS geolocation.
- ALTER TABLE usuario ADD COLUMN longitud DOUBLE NULL
  Added for GPS geolocation.
- ALTER TABLE usuario ADD COLUMN genero VARCHAR(20)NULL
  Added for user profiles

To recreate the database from scratch, the full SQL creation script must be executed in phpMyAdmin.

4. Web application

4.1 Technology used

- PHP 8 with sessions for authentication management
- MVC(Model-View-Controller) design pattern
- JavaScript with fetch()for asynchronous calls
- Bootstrap 5 for styles and responsive components
- cURL for all API requests from PHP
- Visual Studio Code as the development environment

4.2 Project structure

The web application follows a custom MVC pattern whitch a single entry point (index.php) that routes all actions via the ¿action = URL parameter. This approach ensures that every user interaction flows through the controller layer before reaching the API.

```
Web/
├── index.php                    # Single entry point and router
├── controladores/
│   ├── ControladorSesion.php       # Handles login and logout
│   ├── ControladorRegistro.php     # Handles user registration
│   ├── ControladorPerfil.php       # Handles profile editing and account deletion
│   └── ControladorPublicaciones.php # Handles image upload, text posts, feed and deletion
├── modelos/
│   ├── Usuario.php                 # Encapsulates all user-related API calls via cURL
│   └── Publicaciones.php           # Encapsulates all publication-related API calls via cURL
├── vistas/
│   ├── Login.php                   # Login form view
│   ├── Registro.php                # Registration form view
│   ├── Menu.php                    # Main feed view with publications
│   ├── Ajustes.php                 # Settings and account options view
│   ├── EditarPerfil.php            # Profile editing form view
│   └── PublicarTexto.php           # Text post creation view
└── public/
    ├── css/
    │   └── styles.css              # Global styles with blue gradient background
    └── img/
        ├── logos/                  # Application logos
        └── iconos/                 # Toolbar icons (circular buttons)
```

The routing logic in index.php uses a PHP switch statement over the action parameter:

```
switch ($action) {
  case 'login':    require 'vistas/Login.php'; break;
  case 'menu':     require 'vistas/Menu.php'; break;
  case 'validarLogin': $ctrlSesion->iniciarSesion(); break;
  case 'subirImagen':  $ctrlPub->subirImagen(); break;
  // ... more cases
}
```

4.3 API Connection

All HTTP communication with the Java REST API is performed server-side using PHP's cURL library, encapsulated in the model classes(Usuario.php and Publicaciones.php). This architecture means that JavaScript in the browser never calls the Java API directly – it only calls local PHP endpoints which then proxy the request to Tomcat.

This design was a deliberate requirement of the project all API access must go through PHP. The request Flow for loading the publication feed is:

- Browser JavaScript calls fetch('index.php?action=menu')
- PHP's Menu.php view instantiates ControladorPublicaciones and calls obtenerTodas()
- The controller calls Publicaciones->obtener($idUsuario) on the model
- The model makes a cURL GET request to /publicacion/lista/{idUsuario} on the Java API
- The Java API queries MySQL, calculates the user's Camino stage, and returns filtered publications as JSON.
- PHP renders the publications directly in the HTML page (server-side rendering)

For actions triggered by user interaction (such as uploading an image or deleting a post), JavaScript uses fetch() to call a local PHP action, which then calls the API:

- JavaScript calls fetch('index.php?action=subirImagen') with the image as Base64 JSON
- PHP's ControladorPublicaciones->subirImagen() reads the JSON input.
- It calls the Publicaciones model which sends a cURL POST to /publicacion/imagen
- The Java API stores the image and calculates the etapa from the user's GPS coordinates

5. Mobile application

5.1 Technology used

- Android Studio with Java.
- Volley library for asynchronous HTTP requests to the API.
- HttpURLConnection for the custom requests (login,upload image,update user).
- FusedLocationProviderClient for GPS geolocation with LocationCallback.
- RecyclerView with custom adapters for the feed.
- Navigation Component with BottomNavigationView for managing the publication management screen.
- SharedPreferences to persist the user session.

5.2 Project structure

The project follows the standard Android Studio structure. Activities handle full-screen interactions while Fragments manage sub-screens within the publication management Activity.

```
app/
└── src/
    └── main/
        └── java/
            └── com/
                └── example/
                    └── chemin/
                        ├── Activities/
                        │   ├── MainActivity.java        # Login screen. Validates credentials via Api.java
                        │   │                            # and stores session in SharedPreferences.
                        │   ├── principal.java           # Main feed. Loads publications filtered by Camino
                        │   │                            # stage. Requests GPS and updates location on launch.
                        │   ├── gestionPublicacion.java  # Publication management with BottomNavigationView
                        │   │                            # hosting three fragments (Home, Dashboard, Notifications).
```

```
│   ├── ajustesUsuario.java        # User settings: edit profile, close session, delete account.
│   ├── editarPerfil.java          # Profile editing form. Pre-fills current data from API.
│   ├── CrearCuenta.java           # New user registration form.
│   ├── ChatsActivity.java         # Chat list screen (placeholder data, in development).
│   ├── mensajes.java              # Individual chat messages screen (placeholder, in development).
│   └── restaurarContrasenha.java  # Password recovery screen (in development).
│
│   ├── AdaptadorPublicaciones.java # RecyclerView adapter for the feed. Shows delete
│   │                               # button only for the current user's own publications.
│   ├── AdaptadorChats.java        # RecyclerView adapter for chat list.
│   └── AdaptadorMensajes.java     # RecyclerView adapter for messages.
│
│   ├── Publicaciones.java         # Publication data model with multiple constructors.
│   ├── Usuario.java               # User data model, implements Serializable.
│   ├── Chat.java                  # Chat data model (nombre, ultimoMensaje, hora).
│   └── mensaje.java               # Message data model (text content).
│
│   └── Api.java                   # Centralizes all HTTP communication with the REST API.
│                                  # Contains login, updateUser, uploadImage, deleteUser,
│                                  # and updateLocation methods.
│
└── Fragments/
    └── gestionPublicacion/
        └── ui/
            ├── home/
            │   └── HomeFragment.java          # Placeholder fragment (not yet implemented).
```

```
├── dashboard/
│   └── DashboardFragment.java      # Text post creation. Uses Volley to call
│                                   # /publicacion/texto endpoint.
└── notifications/
    └── NotificationsFragment.java  # Placeholder (not yet implemented).
```

5.3 API Connection

The mobile app communicates directly with the Java REST API. The base URL differs depending on the runtime environment:

- Android Emulator: http://10.0.2.2:8080/tema5maven/rest (10.0.2.2 maps to the host machine's localhost)
- Physical device: http://192.168.1.X:8080/tema5maven/rest (where X is the PC's local IP on the shared WIFI network)

The GPS-based filtering workflow works as follows:

- On launch, principal.java calls pedirUbicacion() which checks for ACCESS_FINE_LOCATION permission.
- If granted, FusedLocationProviderClient.requestLocationUpdates() retrieves the current position.
- The coordinates are sent via Volley POST to /usuario/ubicacion, updating the database.
- When cargarPublicaciones() runs, it calls GET /publicacion/lista/{idUsuario}.
- The Java API reads the user's stored coordinates, calculates the nearest Camino stage using Haversine, and returns only publications from that stage and the next.

Publications from other users can only be deleted if their idUsuario matches the logged-in user's ID (checked in AdaptadorPublicaciones.java). Deletion is performed via DELETE /publicacion/eliminar/{id} using Volley's StringRequest.

6. Desktop application

6.1 Technology used

- C# with Windows Forms
- .NET Framework 4.7.2
- HttpClient for HTTP API requests
- Newtonsoft.Json 13.0.4 for JSON deserialization
- Visual Studio 2022

6.2 Project structure

The desktop application is structured around Windows Forms with a static session class and a centralized API class. Unlike the mobile and web apps,the desktop appshows all publications without stage filtering, as it has no access to GPS coordinates.

```
chemin/
├── Form1.cs
├── FormPrincipal.cs
├── FormRegistro.cs
├── FormPerfil.cs
├── FormEnviarImagen.cs
├── FormEnviarTexto.cs
├── TarjetaPublicacion.cs
├── Publicacion.cs
├── Api.cs
├── Sesion.cs
└── Program.cs
```

The project uses a simplified structure where all classes are located in the same directory instead of being separated into multiple folders (such as Forms, Models or Services). This approach was chosen to simplify development and reduce project complexity.

6.3 API Connection

The desktop application uses HttpClient to make direct HTTP calls to the Java REST API. All API methods in Api.cs are async, using async/await to avoid blocking the UI thread during network operations.

A key design decision:since the project targets .NET Framework 4.7.2, System.Text.Json is not available. Two approachesare used depending on the operation:

- Newtonsoft.Json with JsonConvert.DeserializeObject<T>() for deserializing API responses into typed objects (e.g., List<Publicacion>).
- Manual JSON string construction using string interpolation for simple request bodies, avoiding the need for serialization libraries on the request side.

The session lifecycle on the desktop works as follows:

- User logs in via Form1: credentials are posted to /usuario/login, response is parsed with Regex to extract id and nombreCompleto, and stored in the static Sesion class.
- FormPrincipal loads: calls Api.GetPublicaionesAsync() which hits /publicacion/lista (all publications, nostage filter).
- User logs out via FormPerfil: Sesion.Cerrar() resets all fields to null/0, Form1 is shown, and FormPrincipal is closed via Application.OpenForms.

7. Security

7.1 Security mechanisms

Password encryption (BCrypt)

Passwords are not stored in plain text. The jBCrypt library is used, which implements the bcrypt algorithm with automatic salting, increasing security against dictionary and brute-force attacks.

```
String hash = BCrypt.hashpw(contrasenha, BCrypt.gensalt());
BCrypt.checkpw(contrasenhaIntroducida, hashAlmacenado);
```

SQL Injection Prevention

All database queries use PreparedStatements with parameterized parameters, preventing user data from being interpreted as part of the SQL query.

```
PreparedStatement ps = conn.prepareStatement("SELECT * FROM usuario WHERE username = ?");
ps.setString(1, username);
```

XSS (Cross-Site Scripting)

In the web application, htmlspecialchars() is used in all user data outputs to prevent HTML or JavaScript code injection.

CORS

A CORS filter (CORSFilter.java) is implemented in the backend, allowing requests from any origin.

Web Sessions

PHP sessions protect pages that require authentication. Any access without a session will automatically redirect to the login page.

7.2 User and permissions management

- Each user can only delete their own post (verification by user ID)
- Deleting an account deletes all of the user's posts.
- The username is stored in lowercase to avoid duplicates due to capitalization.
- When deleting an account on Android, SharedPreferences are cleared before redirecting to the page.
- The username and email fields have a UNIQUE restriction in the database.

7.3 Good practices

- Secure password storage using BCrypt.
- Use of PreparedStatements to prevent SQL injection.
- Use of htmlspecialchars() on the web to prevent XSS.
- Separation of business logic into a centralized REST API.
- Session control on each client based on its capabilities
- Images are converted to Base64 for secure transmission.

8. Deployment

The deployment of the Chemin project involves four components:the REST API, the web application, the mobile application,and the desktop application. All four must be running for the full system to work. The following sections describe the prerequisites and deployment steps for each component.

8.1 API Deployment

The rest API is developed in Java with Maven and deployed on Apache Tomcat 10.0.27. It can run locally or on a remote server.

Prerequisites

- Java JDK 11 or higher
- Maven installed and configured
- MySQL running on port 3306(XAMPP)
- Database created in phpMyAdmin
- Apache Tomcat 10.0.27

Steps

1. Compile the project with Maven:
   mvn clean package
2. Copy the WAR file to the Tomcat webapps folder:
   Copy-Item .\target\tema5maven.war C:\...\webapps\tema5maven.war -Force
3. Start Tomcat from VS Code or manually.
4. The API will be available at:
   http://localhost:8080/tema5maven/rest

8.2 Web Application Deployment

The web application is built with PHP and server via Apache from XAMPP.No build step is required.

Prerequisites

- XAMPP with Apache and MySQL running
- PHP 8 or higher

Steps

1. Copy the project folder to XAMPP's htdocs:
   C:\xampp2\htdocs\Web\
2. Start Apache from the XAMPP control panel.
3. Access it from your browser:
   http://localhost/Web/index.php

   The application will communicate with the Java API via PHP cURL. Ensure the API is running before opening the web app.

8.3 Mobile Application Deployment

The mobile application is developed for Android and deployed via Android Studio.

Prerequisites

- Android Studio installed
- Emulator or physical device with Android 8.1 or higher
- REST API running and accessible

Steps

1. Open the project in Android Studio.
2. Configure the URL in Api.java according to the environment:

```
// Emulador:          http://10.0.2.2:8080/tema5maven/rest
// Dispositivo fisico: http://192.168.1.X:8080/tema5maven/rest
```

3. Run the application (Run>Run app).

Note: The app requires ACCESS_FINE_LOCATION permission for the GPS-based stage filtering to work. This permission is requested at runtime when the main feed screen is opened.

8.4 Desktop Application Deployment

The desktop application runs natively on Windows and requires Visual Studio 2022.

Prerequisites

- Visual Studio 2022 or later
- .NET Framework 4.7.2
- Running and accessible REST API

Steps

1. Open the solution in Visual Studio 2022.
2. Verify the URL in Api.cs:
   ```
   private const string BASE_URL = "http://localhost:8080/tema5maven/rest";
   ```
3. Compile (Build > BuildSolution).
4. Execute (Ctrl + F5).

8.2 Execution Environments

Around	Description
Local development	All services are on localhost. The Android emulator uses version 10.0.2.2.
Physical device	Mobile and PC on the same WIFI network. Use the PC's local IP address(e.g., 192.168.1.104).
Production	API on a public cloud server. Clients configured with the server URL.

9. Usage and Maintenance Guide

9.1 Running Locally

To run the complete Chemin system locally, start the components in this order:

- Start XAMPP and ensure both Apache and MySQL are running (green status in XAMPP Control Panel)
- Start Tomcat from VSCode. Wait for the Tomcat log to confirm deployment.
- Open the web app in a browser: http://localhost/Web/index.php
- Open the mobile project in Android Studio and run it on an emulator or connected device.
- Open the desktop project in Visual Studio 2022 and run it with Ctrl+F5

9.2 Adding Functionality

To add a new feature, changes must be made consistently across all affected components:

- API: Add a new endpoint method in the appropriate Java class with JAX-RS annotations. Rebuild with mvn clean package and redeploy the WAS to Tomcat.
- Web: Add the new action case to the switch in index.php, create the controller method, add the model method with cURL and create the view file in vistas/.
- Mobile: Create a new Activity or Fragment, declare it in AndroidManifest.xml, add the corresponding method in Api.java, and wire the navigation from the appropriate screen.
- Desktop: Create a new Form, add the method in Api.cs, and add navigation from the relevant form (typically FormPrincipal).

9.3 Common Issues

Problem	Solution
API does not respond	Verify Tomcat is running and the WAR was deployed correctly. Check the Tomcat log for errors.
Web page shows PHP source code	Access the app via XAMPP(http://localhost/..) not by opening the file directly in a browser.

Android emulator cannot connect to API	Use 10.0.2.2 instead of localhost in API.java. The emulator uses this address to reach the host machine.
Physical Android device cannot connect	Ensure the device and PC are on the same WIFI network. Use the PC's local IP (found with ipconfig). Disable any firewall blocking port 8080.
Images not displaying on Android	The image format must be JPEG or PNG.AVIF format (used by some Android cameras) is not supported by Android's BitmapFactory.
Desktop app shows blank feed	Verify the API is running. Check that BASE_URL in Api.cs matches the correct address. Look for errors in the Visual Studio Output window.
After redeploying the WAR	Restart Tomcat for the new versión to take effect. Simply replacing the WAR while Tomcat is running may not be sufficient.
MySQL connection error in API	Verify XAMPP MySQL is running on port 3306.Check the JDBC credentials in GestionaPublicaciones.java and GestionaRegistroUsuario.java

10. Future Improvements

While Chemin already implements the core functionality of a multiplatform social network, there are several planned improvements and new features that would significantly enhance the user experience and make the application production-ready.

10.1 Planned Improvements

- Real-time private messaging between users. The database table (mensaje) is already created. The Android screens (ChatsActivity.java and mensajes.java) are scaffolded. Implementation requires connecting them to real API endpoints.
- Follower system to see posts from specific pilgrims. The database table (seguidor) is already created.
- Like system for publicaciones. The database table (like_publicacion) is already created.
- Repost functionality. The database table (republica) is already created.

- Password recovery via email. The Android screen (restaurar contrasenha.java) is already scaffolded.
- Extending support to all Camino routes, not just the Portuguese Way.

10.2 New Functionalities

- Interactive map showing the real-time location of nearby pilgrims, displayed as pins on a map view.
- Offline mode for areas without mobile coverage, which is common on rural sections of the Camino.
- Stage-based group chat where all pilgrims in the same stage can communicate.
- Integration with local businesses (albergues, restaurants) along the route for recommendations and promotions.
- User search by username or stage.

10.3 Technical Optimizations

- JWT token authentication in the API to replace the current session-based approach. This would make the API stateless and more secure.
- Pagination for the publication feed to improve performance as the number of post grows.
- Image compression in the backend before storing to the database, reducing storage requirements and improving load times.
- Push notifications on the mobile app when new publications appear in the user's current stage.
- Cloud deployment of the API (e.g., AWS or Google Cloud) to make Chemin accessible from anywhere on the Camino, not just a local network.
- Rate limiting on the API endpoints to prevent abuse.
- Stage filtering on the desktop application using the PC's network location as a rough approximation.